

The DMM6500 trigger model — a working manual

Ian Ameline · version 1.41 · MIT licence

The trigger model is not a trigger setting. It is a small programmable machine with its own instruction store, and the instrument's own scanning feature is written in it. This describes what it is, and what it does as opposed to what the reference manual says it does.

Claims marked **MEASURED** were established on this instrument — DMM6500, firmware **1.7.17a** — and each names its experiment. Everything else comes from the *DMM6500 Reference Manual*, DMM6500-901-01 Rev. A/April 2018, cited where the location is pinned down. Anything marked NOT ESTABLISHED has neither a measurement nor a citation and should be read as an open question.

1. The machine

program store	63 blocks , indexed from 1 (§9-49)
sequences	concurrent, each <code>START ... END</code> ; the event log calls them paths
control flow	8 branch types — always, once, once-excluded, on-event, counter, delta, limit-constant, limit-dynamic — plus <code>RESET_BRANCH_COUNT</code> , which resets a counter and does not branch
signals	8 notify blocks, 2 blenders, 4 timers, 6 digital I/O lines, 8 LAN objects, 3 TSP-Link lines
state	<code>trigger.model.state()</code> → <code>EMPTY</code> , <code>BUILDING</code> , <code>RUNNING</code> , <code>WAITING</code> , <code>IDLE</code> , <code>ABORTED</code> , <code>FAILED</code> observed; <code>ABORTING</code> and <code>PAUSED</code> documented but never seen

A program counter, conditional branches, per-block loop counters, and inter-sequence signalling. No general storage and no arithmetic, so it is not a computer — but a rendezvous protocol between two sequences is expressible, and that is what scanning uses.

2. The instruction set

BUFFER_CLEAR	[buffer]	
MEASURE_DIGITIZE	[buffer [, count]]	count: a value, COUNT_INFINITE,
COUNT_STOP, COUNT_AUTO		
WAIT	event [, clear [, logic, event [, event]]]	
DELAY_CONSTANT	seconds	
DELAY_DYNAMIC	delayList, index	
NOTIFY	trigger.EVENT_NOTIFYN	
NOP		
LOG_EVENT	eventNumber, message	
DIGITAL_IO	bitPattern, bitMask	
CONFIG_RECALL	"list" [, index]	
CONFIG_NEXT	"list"	
CONFIG_PREV	"list"	
RESET_BRANCH_COUNT	counter	
BRANCH_ALWAYS	block	
BRANCH_ONCE	block	
BRANCH_ONCE_EXCLUDED	block	
BRANCH_ON_EVENT	event, block	
BRANCH_COUNTER	targetCount, block	
BRANCH_DELTA	targetDifference, block, measureBlock	
BRANCH_LIMIT_CONSTANT	limitType, limitA, limitB, block [, measureBlock]	
BRANCH_LIMIT_DYNAMIC	limitType, limitNumber, block [, measureBlock]	

Three more exist and are **undocumented** — `trigger.BLOCK_START`, `trigger.BLOCK_END` and `trigger.BLOCK_CHANNEL_ACTION` are real TSP constants (MEASURED, E3c) and appear in no `setblock` usage list. A fourth, `WRITE_USB`, is emitted by `scan.export()` and the string appears **zero** times in the reference manual (MEASURED, §9 worked example below).

3. Events

The signal namespace a `WAIT` or a `BRANCH_ON_EVENT` can name:

EVENT_ANALOGTRIGGER	the signal on INPUT HI/LO crossing a comparator level	
EVENT_DISPLAY	the front-panel TRIGGER key	
EVENT_EXTERNAL	rear EXT TRIG IN	
EVENT_COMMAND	a bus trigger (*TRG, GPIB GET, VXI-11 device_trigger)	
EVENT_TIMERN	trigger timer N expired	(1..4)
EVENT_NOTIFYN	notify block N executed	(1..8)
EVENT_BLENDERN	blender N fired	(1..2)
EVENT_DIGION	edge on digital input N	(1..6)
EVENT_LANN	LXI trigger packet on object N	(1..8)
EVENT_TSPLINKN	edge on TSP-Link sync line N	(1..3)
EVENT_SCAN_CHANNEL_READY / EVENT_SCAN_MEASURE_COMPLETE / EVENT_SCAN_COMPLETE /		
EVENT_SCAN_ALARM_LIMIT		
EVENT_NONE	no event — illegal as a WAIT's first event	

Digital I/O, GPIB and TSP-Link need a communications accessory card (KTTI-GPIB, KTTI-TSP, KTTI-RS232).

4. Writing a program

```
trigger.model.load("Empty")           -- empties the store; any load() replaces
it
trigger.model.setblock(1, trigger.BLOCK_BUFFER_CLEAR, buf)
trigger.model.setblock(2, trigger.BLOCK_WAIT, trigger.EVENT_ANALOGTRIGGER,
                        trigger.CLEAR_ENTER, trigger.WAIT_OR, trigger.EVENT_DISPLAY)
trigger.model.setblock(3, trigger.BLOCK_MEASURE_DIGITIZE, buf, 20000)
trigger.model.initiate()              -- returns IMMEDIATELY; the model runs in
firmware
print(trigger.model.getblocklist())    -- disassembles the store
```

setblock DOES NOT CLEAR THE STORE (MEASURED, E1d). It overwrites the blocks you name and leaves the rest in place. Writing a 3-block program over a previous 5-block one leaves blocks 4 and 5 live: in E1d a leftover `BRANCH_COUNTER 1000 → 2` turned a straight-line program into a thousand-iteration loop over the new `WAIT`. **Always `load("Empty")` first.** The name is case-insensitive.

ANY `load()` REPLACES THE STORE WHOLESALE, AND TRUNCATES IT (MEASURED, P4, against eight `NOP` s):

<code>load("SimpleLoop", ...)</code>	5-block template	→ 5 blocks, blocks 6–8 GONE
<code>load("LoopUntilEvent", ...)</code>	6-block template	→ 6 blocks, blocks 7–8 GONE
<code>load("Empty")</code>		→ 0 blocks
<code>trigger.model.abort()</code>		→ 8 blocks, untouched
<code>trigger.clear()</code>		→ 8 blocks, untouched
<code>setblock(1, BLOCK_NOP)</code>		→ 8 blocks, untouched

A short template over a longer program leaves no tail blocks, so there is no stale-tail trap. The accurate rule: **any `load()` replaces the store entirely; `abort()`, `trigger.clear()` and `setblock` do not clear it; `load("Empty")` is the only way to leave it empty.** Note `trigger.clear()` exists and clears trigger EVENT detectors, not the program — the name invites exactly the wrong assumption.

`initiate()` is overlapped. It arms the machine and returns; Lua keeps running, and reading the buffer while the model runs works — MEASURED, §7.

THE MEASURE FUNCTION CANNOT BE CHANGED WHILE THE MODEL RUNS, AND THE REFUSAL IS OUT OF BAND (MEASURED, P2f). With a model genuinely held in `STATE_WAITING`,

`dmm.digitize.func = ...` returned `pcall` true, left the function unchanged, left the state `WAITING`, and posted 2727, **"Command prohibited while trigger model is running"**. Writing the value it already held posted 2727 too, so the refusal is on the write, not on the value.

NOT ESTABLISHED: that switching between local and remote control aborts a running model. It has no measurement here, it is not scriptable from the socket, and the local/remote text at 15-2 says only that the front panel is disabled and that you are prompted to switch control.

5. Neither `setblock` nor `initiate` reports failure

This is the most important thing here, and it is the opposite of what a Lua programmer expects.

call	pcall says	what happened
setblock(64, ...)	true	refused — event 1130 , "Parameter block number, expected value from 1 to 63" (MEASURED, E3a)
setblock(1, BLOCK_WAIT, EVENT_NONE) then initiate()	true, true	state → STATE_FAILED , event 2707 , "Block number 1 has an invalid event ID in parameter 1" (MEASURED, E3b)

| dmm.digitize.func = ... while the model runs | **true** | discarded — event **2727**, "Command prohibited while trigger model is running" (MEASURED, P2f) |

| trigger.model.pause() on a model that is not running | **true** | refused — event **2767**, "Trigger model cannot be paused unless it is already running" (MEASURED, P4) |

THE WHOLE TRIGGER SUBSYSTEM REPORTS REFUSALS OUT OF BAND, not just these two calls: a block write, an ordinary attribute write and a plain function call each return success and file the reason as an event, with its own number. Treat a successful return from anything in `trigger.` as carrying no information about whether the work happened.

A `pcall` verdict on any of them is a constant. **`trigger.model.state()` is the only verdict that means anything**, and the reason lives in `eventlog.next()`. A program that builds and initiates "successfully" can be dead. `setblock(63, ...)` is accepted with no event, so the ceiling is exactly as documented — enforced, but reported out of band.

Nothing in §9 or §15 says this, and no example in the manual checks for it.

A command arriving on the remote interface ABORTS a running model

A model that is waiting cannot be WATCHED from a host. Every command that arrives on the socket while one is initiated aborts it, and the abort is filed as event **2728**, "Trigger model path #1 has been aborted" (MEASURED, `tools/probe_waitstate.py`). The control is what makes it conclusive — one chunk that initiates, delays four seconds **inside the instrument**, and reports:

	state	readings	events
at <code>initiate()</code>	<code>STATE_RUNNING</code>	9	—
4 s later, nothing sent in between	<code>STATE_WAITING</code>	21 100	1, which is 2731 "initiated"
the same program, state polled from the host twice a second	<code>STATE_RUNNING</code> , then <code>STATE_ABORTED</code> within 0.5 s	frozen at a variable count — 4 500, 9 233, 10 645, 11 756, 14 159 over five runs	2731, then 2728

The variable count is the tell: the model dies when the next command arrives, not at a point of its own. Four program shapes behave identically — `LoopUntilEvent` at position 5 and at position 0, and the hand-built three- and two-block forms — so this is the interface and not the program.

SO A WAIT HAS TO BE POLLED FROM INSIDE THE INSTRUMENT, in the same Lua call that initiated it, with `delay()` between looks. That is not a style preference; it is the only way the model survives.

AND THE STATE AT `initiate()` IS NOT THE STATE OF THE WAIT. `STATE_RUNNING` with nine readings already taken is the model executing the blocks AHEAD of its wait — a buffer clear, a zero delay and the start of an infinite digitize. On the 21 100-reading frame buffer `STATE_WAITING` arrives about a millisecond

later. Code that reads "not waiting" as "the event has arrived" is therefore wrong on its first look, and right from the second — but **that millisecond is a property of the buffer, not of the firmware**, and on a recording it is hundreds of times longer. §8 has the scaling and what it cost.

AND KEITHLEY'S OWN HOST CODE RELIES ON THE OPPOSITE, WHICH IS NOT YET RECONCILED.

`Instrument_Examples/DMM6500/Streaming_Examples/00_Stream_Data_from_DMM6500/Stream_DMM6500.py` queries an initiated model over a **raw socket on port 5025**, in a loop, for the whole length of a run, and depends on the model continuing: its `get_data()` blocks inside the instrument until the buffer grows, so an abort on the first query would deadlock the host. It never calls `trigger.model.abort()` at all.

The obvious explanation — that a measuring model survives where a waiting one does not — is refuted by the table above. Readings were still accumulating when each abort landed, 4 500 to 14 159 of them, so those models were digitizing too. Whatever separates the two cases, it is not the block type. Two differences remain open: that script writes into `defbuffer1` with `FILL_CONTINUOUS` rather than into a made buffer, and its reads are `printbuffer` calls rather than state polls. **Unresolved.** Until it is settled, the rule above stands as measured, because it was measured on this instrument and the counter-example was not.

One caution on that counter-example, since it is easy to over-read: of the three vendor scripts that look like evidence here, one (`KEIDMM6500_Stream_Measured_Dual_Meter.py`) talks **USBTMC** through PyVISA, not a socket, and pairs a DMM6500 with a DAQ6510 — so it says nothing about this interface.

`Stream_DMM6500.py` is the only raw-socket DMM6500 case of the three.

6. BLOCK_WAIT : three events, and a latch

```
setblock(n, trigger.BLOCK_WAIT, event [, clear [, logic, event [, event]]])
```

`logic` is `trigger.WAIT_OR` or `trigger.WAIT_AND` and governs the whole list — you cannot mix them in one block (§15-413). Three events in one block means **a blender is unnecessary for a simple OR**: "the analog trigger, OR the front key, OR the rear BNC" is one block (MEASURED, E1c).

The wait latches. An event that fires before execution reaches the block is remembered, and `clear` decides what happens:

<code>clear</code>	an event that already fired	MEASURED (E3g)
<code>trigger.CLEAR_NEVER</code> (default)	passes straight through	state <code>IDLE</code> , readings taken
<code>trigger.CLEAR_ENTER</code>	discarded; waits for a fresh one	state <code>WAITING</code> , 0 readings

The default is the dangerous one. A capture armed on the TRIGGER key with `CLEAR_NEVER` fires on a press made minutes earlier. **Use `CLEAR_ENTER` for anything a human can trigger.** Use `CLEAR_NEVER` deliberately when you want a semaphore — see §9. The latch also clears at the start block and on exiting the wait.

THE DEFAULT `logic` IS `AND` , NOT `OR` . A wait built as `(event, clear)` with no logic argument disassembles as `LOGIC: AND` , while `LoopUntilEvent` emits `LOGIC: OR` . Harmless for one event, and a trap the moment a second is added.

7. Pre-trigger is a program shape, not a setting

Digitize infinitely into a circular buffer, wait, then digitize a counted burst. The counted block stops the infinite one.

```
1) BUFFER_CLEAR      buf
2) MEASURE_DIGITIZE  buf, trigger.COUNT_INFINITE  -- fills continuously, wraps
3) WAIT              <event>, CLEAR_ENTER
4) MEASURE_DIGITIZE  buf, <post-trigger count>    -- stops the infinite block
```

MEASURED (E3d). 1 kSa/s into a 5000-reading buffer, a timer firing the wait at 2 s, post-trigger count 500: `buf.n` read 503 at ~0.5 s, 1505 at ~1.5 s, **2501** final — 2001 pre-trigger readings plus 500 post. The buffer accumulates *across* the wait.

THE PRE-ROLL IS NOT OPTIONAL: AN ANALOG-TRIGGER WAIT CANNOT BE THE FIRST BLOCK (MEASURED, P3). Drop the `COUNT_INFINITE` block and the program becomes

```
1) WAIT              EVENT_ANALOGTRIGGER, CLEAR_ENTER
2) MEASURE_DIGITIZE  buf, 400
```

which waits for ever. Three seconds on a live 1 kHz square wave, comparator armed dead centre of its 0-3 V swing, gave `STATE_WAITING` with **zero readings**, no event and no error. The control is decisive: the same level, the same slope, the same signal and the same session fired in **under 0.01 s** once a `COUNT_INFINITE` block was put ahead of the wait.

EVENT_ANALOGTRIGGER is derived from the digitizer's sample stream, so the comparator only evaluates while a measure/digitize block is running. With the wait first, nothing is sampling, the comparator never sees the signal, and the wait cannot be satisfied. There is no diagnostic — the model simply sits, which is indistinguishable from a trigger that has not arrived yet.

This is a trap worth stating plainly because the broken program looks *better* than the working one: it is two blocks instead of four and it is on the fast path (§10). It is on the fast path because it never does anything.

NOT ESTABLISHED: whether a wait on `EVENT_DISPLAY` or `EVENT_EXTERNAL` as the first block works. Neither depends on the sample stream, so both may be fine; the restriction established here is specific to the analog trigger. Also not established is whether a *finished* counted pre-roll suffices or whether the acquisition must be concurrently active.

THE WAIT ALSO KEEPS THE PRE-ROLL RUNNING, not merely awaits the trigger (MEASURED, P2f).

An infinite digitize as the *only* block idles after about a dozen samples whatever `dmm.digitize.count` says, because the sequence ends and that stops the block. With the wait downstream the same block filled a 20 000-reading buffer and sat in `WAITING`. The counted block stops the infinite one; the wait is what lets it run at all.

The window therefore contains everything from `initiate()` onward, so a capture armed this way cannot miss the start of the signal it waits for. Pre-trigger depth is bounded by buffer capacity, not by a percentage.

THE COMPARATOR'S LEVEL AND SLOPE ARE BOTH HONOURED (MEASURED, P2, against a 1 kHz 0-3 V square wave). Armed at 8 V — outside the signal's range — the model sat in `STATE_WAITING` indefinitely and never fired, so the level is real and not a formality. Armed at mid-swing, reading either side

of the trigger index: `SLOPE_RISING` gave -0.001 V before and 3.000 V after, `SLOPE_FALLING` gave 3.000 V before and -0.001 V after, each with a single partial-transition sample at the edge. Opposite directions from the two settings.

An explicit count on the block **overrides** `dmm.digitize.count` (MEASURED, E3e: count 7 set, block count 250, 250 readings taken). Counts of 8192, 32768 and 1 000 000 are all accepted (MEASURED, E3f).

8. The canned templates, disassembled

`LoopUntilEvent(event, position, clear, delay, buffer)` — the shape above (MEASURED, E1a):

```
1) BUFFER_CLEAR  2) DELAY_CONSTANT 0  3) MEASURE_DIGITIZE INFINITE
4) WAIT <event> CLEAR:ENTER           5) MEASURE_DIGITIZE <count>  6) NOTIFY 2
```

`position` is a pre-trigger **percentage**, and the post-trigger count the template programs is

```
count = (100 - position)% of the BUFFER'S CAPACITY
```

`dmm.digitize.count` **has no part in it**. MEASURED, E4: capacity and position varied independently across 15 combinations, each matching the rule exactly, and `dmm.digitize.count` set to 77 against 1000 changed nothing.

capacity	position 0	5	20	50
10 000	10 000	9 500	8 000	5 000
20 000	20 000	19 000	16 000	10 000
50 000	50 000	47 500	40 000	25 000

SO AN ARMED CAPTURE NEVER FILLS ITS BUFFER, AND THE SHORTFALL IS NOT A FAULT. At

`position` 5 the template programs 95 % of capacity as the post-trigger count, and the readings above that come from the pre-trigger phase — however many accumulated between `initiate()` and the trigger arriving. On a continuously active line that is a handful of samples, not the 5 % the position nominally reserves, so a completed capture lands a little over 95 % of capacity and never at 100 %. Testing for a full buffer as the completion condition therefore fails on every capture that worked; `trig_done()` is the test that holds.

BLOCK 1 RUNS OVER THE WHOLE BUFFER, SO THE TIME TO REACH THE WAIT SCALES WITH THE RECORDING. The three blocks ahead of the wait are not free, and the first of them clears every slot. Buffer work at this size is linear in readings: `buffer.make()` measures **0.0019 s at 21 100** readings and **0.7789 s at 2 800 000**, which is **0.28 μ s each**. So the interval between `initiate()` and `STATE_WAITING` is a function of capacity, not a constant:

the capture	capacity	setup, at 0.28 μ s a reading
a frame capture	21 100	~0.006 s — measured arrival about 1 ms
an 8 kB recording	~862 000	~ 0.24 s
a 32 kB recording	2 800 000 (the <code>ck_bufmax</code> ceiling)	~ 0.78 s

A FIXED BOUND MEASURED ON THE FRAME BUFFER IS THEREFORE WRONG FOR A RECORDING, AND WRONG AS A COIN TOSS RATHER THAN CLEANLY. `sdec.arm_settle_s` bounded that spin at

0.25 s — forty times the frame path's millisecond — against an 8 kB recording's 0.24 s of setup.

MEASURED, `hw-arm` case F: 0.25 s fails, including on a completely fresh load, while 0.6 s, 1.0 s and 3.0 s each arm and collect about 5 484 bytes. The same case passed once and failed the next three runs at 0.25.

The consequence inverts the feature instead of degrading it. A spin that never sees `STATE_WAITING` has to report an arm that did not go live, because the poll loop cannot tell that from a trigger that fired — so the recording runs free and captures exactly the silence that arming exists to avoid.

AND A TIGHT POLL LOOP APPEARS TO STARVE THE TRANSITION. A probe reading

`pcall(trigger.model.state)` as fast as Lua can issue it — 200 000 reads in 10 s, **0.014 ms** each — reported `STATE_RUNNING` for the full bound at **every** capacity including 21 100, while the app sees `STATE_WAITING` within milliseconds polling every 10 ms. `buf.n` reached capacity throughout, so the model was acquiring. Treat a state poll as something to do at the app's cadence, not flat out.

`SimpleLoop(count, delay, buffer)` — a counted loop with **no wait at all** (MEASURED, E1b):

```
1) BUFFER_CLEAR  2) DELAY_CONSTANT 0  3) MEASURE_DIGITIZE <buf>, 1
4) NOTIFY 2      5) BRANCH_COUNTER <count> → 2
```

`SimpleLoop` acquires the instant it is initiated. A recording built on it **cannot** be armed, and a 32 768-sample recording executes 32 768 iterations of a three-block loop.

9. A worked example: the firmware's own scan program

The best teacher is the program the instrument writes for itself. A 6-channel RTD scan —

`scan.add("1,6,2,3,4,5")`, `scancount` 10000, `scaninterval` 10 s, `scan.export(..., WRITE_AT_END)`, on a `2000,10-Chan Mux` — generates **29 blocks in two sequences** (MEASURED):

```

A 1) START
   2) CONFIG_RECALL ChannelList INDEX 1      -- the INIT step: open all
   3) CHANNEL_ACTION
   4) WAIT EVENT_TIMER4 CLEAR:NEVER           -- the interval gate
   5) NOTIFY 5                                -- restarts timer 4
   6) CONFIG_RECALL ChannelList INDEX 3
   7) BRANCH_ONCE → 11                        -- cold start: no prior measurement to
wait for
   8) BRANCH_ALWAYS → 10
   9) CONFIG_NEXT ChannelList
  10) WAIT EVENT_NOTIFY7 CLEAR:NEVER          -- "the measurement finished"
  11) CHANNEL_ACTION                          -- close the next channel
  12) NOTIFY 6                                -- "the channel is ready"
  13) BRANCH_COUNTER 6 → 9                    -- inner loop: 6 channels
  14) BRANCH_COUNTER 10000 → 4                -- outer loop: 10000 sweeps
  15) END

B 16) START
   17) BUFFER_CLEAR defbuffer1
   18) WAIT EVENT_TIMER4 CLEAR:NEVER          -- the same interval gate
   19) CONFIG_RECALL MeasureList INDEX 3
   20) BRANCH_ALWAYS → 22
   21) CONFIG_NEXT MeasureList
   22) WAIT EVENT_NOTIFY6 LOGIC:AND           -- "the channel is ready"
   23) MEASURE_DIGITIZE defbuffer1 COUNT:AUTO
   24) NOTIFY 7                               -- "the measurement finished"
   25) BRANCH_COUNTER 6 → 21
   26) NOTIFY 8                               -- "the sweep finished"
   27) BRANCH_COUNTER 10000 → 18
   28) WRITE_USB defbuffer1                  -- scan.export, WRITE_AT_END
   29) END

```

Five idioms worth stealing, none of them in the reference manual:

A ping-pong rendezvous. A closes a relay and raises `NOTIFY 6`; B waits on it, measures, raises `NOTIFY 7`; A waits on that before advancing. Neither sequence can outrun the other. `CLEAR:NEVER` is deliberate here and is what makes a notify behave as a semaphore rather than a race — a notify raised before the partner reached its wait is still honoured.

A self-restarting interval timer. `trigger.timer[4]` has `delay = 10`, `count = 1` and `start.stimulus = trigger.EVENT_NOTIFY5`. Blocks 4 and 18 wait on `EVENT_TIMER4`; block 5 raises `NOTIFY 5`, which starts the timer. Each sweep therefore re-arms the clock at its own start, giving a period measured sweep-start to sweep-start rather than 10 s of dead time added to a sweep of unknown length. The timer configuration lives outside the trigger model, in the setup script.

Loop peeling with `BRANCH_ONCE`. Block 7 skips the `WAIT NOTIFY7` exactly once, because on the first channel of the first sweep there is no earlier measurement to wait for. Every later pass falls through to block 8 and waits properly. One block replaces a prologue.

Configuration lists as the data plane. Per-channel settings are not expressed as branches. Two lists — `ScanAutoCreatedChannelList` and `ScanAutoCreatedMeasureList` — are stepped by `CONFIG_NEXT` inside each inner loop, and `MEASURE_DIGITIZE` uses `COUNT:AUTO` to take each channel's own count from the recalled configuration. The program is the control flow; the lists are the parameters.

No blenders. Both are untouched — `orenable` false, every stimulus `EVENT_NONE` . Keithley synchronises with notifies and timers alone. For a simple OR of two trigger sources a multi-event `WAIT_OR` is the idiomatic answer, and it costs no blender at all.

Two cautions this example also demonstrates. `WRITE_AT_END` puts the USB write in block 28, so a power loss anywhere in the preceding 27.8 hours costs the entire record. And `scan.monitor.channel` set without `scan.monitor.mode` generates **no** limit or branch-on-limit blocks at all: the monitor is inert and the scan simply runs on its interval.

The first sweep stalls for one whole interval, and the two steps that establish it are worth keeping because the conclusion is not obvious from the listing. A timer carrying a `start.stimulus` does **not** start when it is enabled: MEASURED (A13) — `timer[1]` with `delay = 1.0` , `start.stimulus =` `EVENT_NOTIFY5` and `enable = ON` , with a program waiting on `EVENT_TIMER1` , sat in `STATE_WAITING` with 0 readings three seconds later. Nothing therefore raises `EVENT_TIMER4` until block 5 runs, and block 5 is downstream of the wait at block 4. `CLEAR:NEVER` cannot rescue it either, because a latch needs an expiry to latch and there has been none.

So the scan takes `scaninterval` seconds to produce its first reading. On a 27.8-hour run that is invisible, which is presumably why it has never been worth fixing.

10. Performance, and the oracle the firmware gives you

`getblocklist()` ends with `* This trigger model may not be set up for low latency. *` whenever the program is off the fast path. That makes the boundary observable, and it is **a list of specific disqualifiers, not a size heuristic** (MEASURED, `verify_latency`).

What costs the fast path:

disqualifier	measured boundary
measure/digitize blocks	1 and 2 keep it; 3 loses it
total blocks	14 keeps it; 15 loses it
<code>BUFFER_CLEAR</code>	any , even one
<code>DELAY_CONSTANT</code>	0, 0.1 and 0.25 s keep it; 0.3 s loses it — the boundary lies in 250-300 ms
<code>LOG_EVENT</code>	any , with a valid event number
two blocks waiting on the same event	any — see below
a configuration-list block (<code>CONFIG_RECALL</code> / <code>NEXT</code> / <code>PREV</code>)	any , even one
a branch to a block that does not exist	any

What is free: `NOP` , `NOTIFY` , `WAIT` (including a 3-event `WAIT_OR`) , `BRANCH_COUNTER` , `BRANCH_ALWAYS` **to a target that exists**, `COUNT_INFINITE` , a second measure block writing to a **different buffer**, and **any number of** `DELAY_CONSTANT` **blocks** — six were free (MEASURED, P4), so the delay disqualifier is duration and not count.

THE FIRMWARE STATES THE MEASURE-BLOCK LIMIT ITSELF. Building a third measure block posts

2753: Trigger model is not optimized for speed due to greater than 2 measure/digitize blocks

which is a primary source for the boundary and better evidence than the sweep that found it.

TWO WAITS ON ONE EVENT IS A REAL DISQUALIFIER, and it is the row most easily mistaken for something else (MEASURED, P1e). Holding the program shape fixed and varying only the events:

one WAIT on EVENT_DISPLAY, then a measure	LOW LATENCY
two WAITs, both on EVENT_DISPLAY	warned
two WAITs, on EVENT_DISPLAY and EVENT_EXTERNAL	LOW LATENCY

It holds at two different block counts, so the cost is the shared event and not the extra blocks. This is also why a WAIT / MEASURE / WAIT / MEASURE program warns while WAIT itself is free — an earlier reading of that program wrongly suspected WAIT.

BRANCH_ALWAYS COSTS NOTHING; A DANGLING TARGET COSTS EVERYTHING (MEASURED, P1a). A BRANCH_ALWAYS to a block that exists is free, backwards or forwards; the same opcode pointing past the end of the program warns. So the disqualifier is the unresolvable target, not the jump.

A REJECTED BLOCK READS AS A FREE ONE, and that is a trap in measuring any of this. LOG_EVENT first measured as free because the probe passed a bare 1 as the event number: setblock returned true, the block was never added, and a program of one measure block is of course on the fast path. The refusal was event 2725, "Log Event block number 2 Event Number is invalid", visible only in the log. With trigger.LOG_INF01 (2734) the block is accepted and the warning appears. **Confirm a block is present in getblocklist() before drawing any conclusion from its absence of effect** — see §5.

THE MANUAL NEVER USES THE WORDS "LOW LATENCY" — zero occurrences in 55 488 lines. The firmware's vocabulary for its own fast path is undocumented, and the manual nowhere connects the flag to its performance advice. That advice is a list of SEVEN bullets at §9-50, headed "Improving the performance of a trigger model". Four of them turn out to be exactly what the flag enforces and three are wrong as written:

§9-50 says	measured
fewer than 15 blocks	agrees — 14 keeps the flag, 15 loses it
constant delays under 254 ms	agrees — 0.25 s keeps it, 0.3 s loses it
four or fewer measure/digitize blocks	stricter than documented — only 2 keep it
do not use multiple reading buffers	contradicted — a second buffer costs nothing
four or fewer delay blocks	not enforced — six were free
no two blocks waiting on the same event	agrees — and it is enforced, not advice
limit configuration-list blocks	agrees, absolutely — one block is enough to lose the flag

THE PRACTICAL CONSEQUENCE IS THE OPPOSITE OF WHAT IT LOOKS LIKE. The two armed shapes most worth building are both on the fast path:

MEASURE_DIGITIZE INFINITE / WAIT / MEASURE_DIGITIZE n	LOW LATENCY	(pre-trigger)
WAIT / MEASURE_DIGITIZE n	LOW LATENCY	(armed
recording)		

while `LoopUntilEvent` and `SimpleLoop` are **not** — and the only reason is the `BUFFER_CLEAR` block each template emits. Clearing the buffer from Lua with `buf.clear()` before `initiate()` instead of with a block keeps an otherwise identical program on the fast path. Every scan is off it too; the 29-block example above exceeds the 14-block limit on its own.

The disqualifiers have a shape, though a weaker one than it first appears. `BUFFER_CLEAR` is a side-effecting operation, a long delay blocks, a dangling branch target cannot be resolved, and two blocks waiting on one event cannot be ordered without knowing which consumes it. A plain unconditional jump, a counted branch and a wait are all free. That is consistent with a fast path that resolves a bounded program ahead of time and falls back to general interpretation when it cannot — but **the mechanism is not stated anywhere in the manual and this document does not claim to have established it**. An earlier draft offered "`BRANCH_ALWAYS` is an unconditional jump" as evidence for that shape; measurement refuted it, and the example is gone.

11. Documentation defects found

- **Notify count, given inconsistently nine times against two.** The event tables say "Notify trigger block N (1 to 3)" at ref lines 13930, 13962, 18230, 18646, 19044, 25468, 26845, 27086 and 27417, and "1 to 8" at only 25774 and 26607. The wrong value is the one that dominates. **Eight is right:** `trigger.EVENT_NOTIFY8` resolves on the instrument, and the scan generator emits `NOTIFY 6`, `7` and `8`.
- **Out-of-band failure reporting is undocumented.** Nothing states that `setblock` and `initiate` report refusals as event-log entries while returning success, and no example checks for it.
- **A constant that does not exist.** The SCPI mapping table at ref line 13860 gives the paused state as `trigger.STATE_PAUSE`. That constant is **nil** on the instrument; the real one is `trigger.STATE_PAUSED`, as ref line 49484 has it.
- **Four undocumented block types.** `START`, `END` and `CHANNEL_ACTION` exist as TSP constants and appear in no `setblock` usage list. `WRITE_USB` appears in the manual not at all.

12. Choosing a program

To capture on a signal edge and keep what preceded it, use §7's shape: an infinite digitize, a wait, then a counted burst. The window then contains everything from `initiate()` onward, so the start of the signal cannot be missed. `LoopUntilEvent` is the canned equivalent and costs the fast path.

To arm a long acquisition on the analog trigger, use §7's four-block shape. **Do NOT reduce it to a wait followed by a counted `MEASURE_DIGITIZE`** — that program builds cleanly, passes every static check, sits on the fast path, and never fires. See the warning below.

To clear the buffer, call `buf.clear()` from Lua before `initiate()` rather than using a `BLOCK_BUFFER_CLEAR`. The block is a fast-path disqualifier and the Lua call is not.

To OR two or three trigger sources, put them in one `WAIT` with `trigger.WAIT_OR` instead of configuring a blender. Fewer blocks, no shared subsystem to release afterwards.

For anything a human can trigger, use `trigger.CLEAR_ENTER`. The default `CLEAR_NEVER` honours a press made before the block was reached.

Read `trigger.model.state()` after every build and `initiate()`, and drain `eventlog.next()`. Neither call reports its own refusal.